Mini Project 2

Spark Case Study: Steam Reviews Analytics

Course: DSAI 427 — Big Data Analytics

Student Name	Student ID
Seif El Din Mahdy Abdelwahab	202200990
Yusuf Tamer Sahab	202201929

April 2026

Problem Statement

Analytical Problem

The Steam gaming platform hosts one of the largest collections of user-generated game reviews in the world. Each review encapsulates not only a user's opinion, but also rich behavioral signals: how many hours they invested in the game, whether their peer community validated their review as helpful, and when they chose to write it. Understanding what makes a review influential — and which games attract the most deeply engaged communities — is a problem with direct relevance to game developers, publishers, and platform recommendation systems.

This project investigates the Steam Reviews dataset to answer the following research questions:

RQ1. Which games generate the most engaged reviewers, measured by playtime and helpfulness scores?

RQ2. What differentiates positive from negative reviews in terms of player behavioral patterns?

RQ3. Which games have the single most influential review, and what characteristics does it share?

RQ4. How does review activity trend over time for individual games, and are there seasonal effects?

RQ5. Are elite reviewers — those with high playtime, high peer validation, and substantive text — disproportionately concentrated in above-average quality games?

Why Distributed Processing Is Required

The dataset contains 1,460,621 rows with 13 mixed-type columns including free-text review content, Unix timestamps, floating-point scores, and large integer identifiers. Answering the research questions requires multi-dimensional groupBy aggregations, window functions over millions of rows partitioned by game, join operations across tables, and repeated scans for subquery logic. A single-machine pandas workflow would face three fundamental bottlenecks: insufficient memory for the string-heavy dataset, prohibitively slow window function computation without executor-level parallelism, and no mechanism for predicate pushdown into columnar file formats. Apache Spark with its Catalyst optimizer, Tungsten execution engine, and distributed shuffle infrastructure is the appropriate tool for this scale and query complexity.

Dataset Description

Source and Size

- **Source:** Kaggle — `forgemaster/steam-reviews-dataset`
- **File used:** `reviews-1-115.csv`
- **Total rows:** 1,460,621
- **Storage format:** CSV (source); converted to Parquet for all query execution

Schema

Table 1: Dataset Schema

Column	Spark Type	Nullable	Description
steamid	LongType	No	Unique Steam user identifier
appid	IntegerType	No	Steam game identifier
voted_up	BooleanType	Yes	True = positive recommendation
votes_up	IntegerType	Yes	Users who marked review helpful
votes_funny	IntegerType	Yes	Users who marked review funny
weighted_vote_score	FloatType	Yes	Steam helpfulness score (0–1)
playtime_forever	IntegerType	Yes	Total minutes played (all time)
playtime_at_review	IntegerType	Yes	Minutes played at time of review
num_games_owned	IntegerType	Yes	Number of games user owns
num_reviews	IntegerType	Yes	Total reviews written by user
review	StringType	Yes	Full text of the review
unix_timestamp_created	LongType	Yes	Unix epoch of review creation
unix_timestamp_updated	LongType	Yes	Unix epoch of last update
review_year	IntegerType	–	Derived: year from timestamp
review_month	IntegerType	–	Derived: month from timestamp

Justification for Distributed Processing

The dataset satisfies all criteria for distributed processing with Apache Spark:

- **Scale:** 1.46 million records with 13 columns including a free-text string field (review body) that creates significant memory pressure when loaded into a single JVM heap.
- **Mixed types:** The schema combines LongType identifiers, IntegerType counts, FloatType scores, BooleanType flags, StringType text, and derived temporal columns — exercising Spark's type coercion and Catalyst's type inference across all query types.
- **Window computations:** Ranking and moving average operations (Q5, Q6) require partitioning data by `appid` and sorting within each partition — an operation that

maps naturally to Spark's Exchange and Sort physical operators but is computationally intractable in single-threaded pandas.

- **Shuffle volume:** Multi-key groupBy operations (Q3) and join operations (Q8, Q10) generate significant inter-executor data movement that benefits from Spark's AQE-managed shuffle infrastructure.
- **File format:** The original CSV was converted to Parquet, enabling columnar reads, predicate pushdown to row-group level, and metadata-based count operations — all impossible with CSV.

Queries Implemented

Each of the 10 queries was implemented in three APIs: DataFrame, Spark SQL, and RDD. The following subsections present the goal, query type, and core code for each implementation.

Query 1 — Complex Filter (Elite Reviews)

Goal: Isolate high-quality "elite" reviews using four simultaneous filter conditions.
Type: Complex Filter
Result: 814 elite reviews identified out of 1.46M (0.056%). Cached for reuse in Q7.

DataFrame API

```
elite_df = (
    reviews
    .filter(F.col("playtime_forever") >= 3_000)
    .filter(F.col("weighted_vote_score") >= 0.7)
    .filter(F.col("voted_up") == True)
    .filter(F.length("review") >= 100)
    .select("steamid", "appid", "voted_up", "votes_up",
            "weighted_vote_score", "playtime_forever",
            "review", "unix_timestamp_created", "review_year")
)
elite_df.cache()
print(f"Elite reviewers: {elite_df.count():,}")
```

Spark SQL

```
CREATE OR REPLACE TEMP VIEW elite_reviews AS
SELECT steamid, appid, voted_up, votes_up, weighted_vote_score,
       playtime_forever, review, unix_timestamp_created,
   review_year
```

```sql
FROM    reviews
WHERE   playtime_forever      >= 3000
  AND   weighted_vote_score   >= 0.7
  AND   voted_up              = true
  AND   LENGTH(review)        >= 100;

SELECT COUNT(*) AS elite_count FROM elite_reviews;
```

RDD API

```python
elite_rdd = (
    reviews_rdd
    .filter(lambda r: r["playtime_forever"] is not None
                      and r["playtime_forever"] >= 3000)
    .filter(lambda r: r["weighted_vote_score"] is not None
                      and r["weighted_vote_score"] >= 0.7)
    .filter(lambda r: r["voted_up"] == "True")
    .filter(lambda r: r["review"] is not None
                      and len(r["review"]) >= 100)
)
print(f"Elite reviewers: {elite_rdd.count():,}")
```

Query 2 — Aggregation (Per-Game Statistics)

Goal: Compute average helpfulness, review count, average playtime, and total helpful votes per game.
Type: Aggregation
Filter: Only games with at least 50 reviews are included.

DataFrame API

```python
per_game_stats_df = (
    reviews
    .groupBy("appid")
    .agg(
        F.round(F.avg("weighted_vote_score"), 4).alias("
   avg_helpfulness"),
        F.count("*").alias("total_reviews"),
        F.round(F.avg("playtime_forever"), 2).alias("
   avg_playtime_mins"),
        F.sum("votes_up").alias("total_votes_up"),
    )
```

```
    .filter(F.col("total_reviews") >= 50)
)
per_game_stats_df.show(10, truncate=False)
```

Spark SQL

```
SELECT appid,
       ROUND(AVG(weighted_vote_score), 4) AS avg_helpfulness,
       COUNT(*)                           AS total_reviews,
       ROUND(AVG(playtime_forever), 2)    AS avg_playtime_mins,
       SUM(votes_up)                      AS total_votes_up
FROM   reviews
GROUP  BY appid
HAVING COUNT(*) >= 50
ORDER  BY total_reviews DESC;
```

RDD API

```
game_stats_rdd = (
    reviews_rdd
    .filter(lambda r: r["appid"] is not None)
    .map(lambda r: (r["appid"], (
        float(r["weighted_vote_score"] or 0),
        int(r["playtime_forever"] or 0),
        int(r["votes_up"] or 0), 1)))
    .reduceByKey(lambda a, b: (
        a[0]+b[0], a[1]+b[1], a[2]+b[2], a[3]+b[3]))
    .filter(lambda kv: kv[1][3] >= 50)
    .map(lambda kv: {
        "appid": kv[0],
        "avg_helpfulness": round(kv[1][0]/kv[1][3], 4),
        "total_reviews":   kv[1][3],
        "avg_playtime":    round(kv[1][1]/kv[1][3], 2),
        "total_votes_up":  kv[1][2]
    })
)
for row in game_stats_rdd.take(10):
    print(row)
```

Query 3 — Multi-Attribute Grouping

Goal: Group by both game and sentiment to compare review behavior between positive and negative reviewers.
Type: Multi-Attribute Grouping
Top result: appid=730 (CS:GO) with 859,030 positive reviews.

DataFrame API

```
by_game_sentiment_df = (
    reviews
    .filter(F.col("voted_up").isNotNull())
    .groupBy("appid", "voted_up")
    .agg(
        F.count("*").alias("review_count"),
        F.round(F.avg("weighted_vote_score"), 4).alias("
   avg_helpfulness"),
        F.round(F.avg("playtime_forever"), 2).alias("
   avg_playtime_mins"),
        F.round(F.stddev("weighted_vote_score"), 4).alias("
   score_std_dev"),
    )
    .orderBy(F.col("review_count").desc())
)
by_game_sentiment_df.show(20, truncate=False)
```

Spark SQL

```
SELECT appid,
       voted_up,
       COUNT(*)                                AS review_count,
       ROUND(AVG(weighted_vote_score), 4)      AS avg_helpfulness
   ,
       ROUND(AVG(playtime_forever), 2)         AS
   avg_playtime_mins,
       ROUND(STDDEV(weighted_vote_score), 4) AS score_std_dev
FROM   reviews
WHERE  voted_up IS NOT NULL
GROUP  BY appid, voted_up
ORDER  BY review_count DESC;
```

RDD API

```
sentiment_rdd = (
    reviews_rdd
    .filter(lambda r: r["voted_up"] is not None)
    .map(lambda r: ((r["appid"], r["voted_up"]), (
        float(r["weighted_vote_score"] or 0),
        int(r["playtime_forever"] or 0), 1)))
    .reduceByKey(lambda a, b: (a[0]+b[0], a[1]+b[1], a[2]+b[2])
   )
    .map(lambda kv: {
        "appid":    kv[0][0],
        "voted_up": kv[0][1],
        "count":    kv[1][2],
        "avg_help": round(kv[1][0]/kv[1][2], 4),
        "avg_play": round(kv[1][1]/kv[1][2], 2),
    })
    .sortBy(lambda r: -r["count"])
)
for row in sentiment_rdd.take(20):
    print(row)
```

Query 4 — Sorting and Ranking (Top Games by Engagement)

Goal: Rank games by a composite engagement score that weights review volume, helpfulness, and log-scaled playtime.

Type: Sorting and Ranking

Formula: `engagement_score = total_reviews * avg_helpfulness * log1p(avg_playtime_mins)`

Top game: appid=730 with engagement score 2,062,600.35.

DataFrame API

```
top10_games_df = (
    per_game_stats_df
    .withColumn(
        "engagement_score",
        F.round(
            F.col("total_reviews") * F.col("avg_helpfulness")
            * F.log1p("avg_playtime_mins"), 2
        )
    )
    .orderBy(F.col("engagement_score").desc())
    .limit(10)
```

```
)
top10_games_df.show(10, truncate=False)
```

Spark SQL

```sql
SELECT appid, avg_helpfulness, total_reviews,
       avg_playtime_mins, total_votes_up,
       ROUND(total_reviews * avg_helpfulness
             * LN(avg_playtime_mins + 1), 2) AS
   engagement_score
FROM (
    SELECT appid,
           ROUND(AVG(weighted_vote_score), 4) AS
   avg_helpfulness,
           COUNT(*)                           AS total_reviews
   ,
           ROUND(AVG(playtime_forever), 2)     AS
   avg_playtime_mins,
           SUM(votes_up)                      AS
   total_votes_up
    FROM   reviews
    GROUP  BY appid
    HAVING COUNT(*) >= 50
) g
ORDER  BY engagement_score DESC
LIMIT  10;
```

RDD API

```python
import math
top10_rdd = (
    game_stats_rdd
    .map(lambda r: dict(r, engagement_score=round(
        r["total_reviews"] * r["avg_helpfulness"]
        * math.log1p(r["avg_playtime"]), 2)))
    .sortBy(lambda r: -r["engagement_score"])
    .take(10)
)
for row in top10_rdd:
    print(row)
```

Query 5 — Window Function: Ranking

Goal: Identify the single most influential review per game using a window rank on helpfulness score and helpful vote count.
Type: Window Function (Ranking)
Result: 96 distinct games with a top-ranked review identified.

DataFrame API

```
influence_window = (
    Window
    .partitionBy("appid")
    .orderBy(F.col("weighted_vote_score").desc(),
             F.col("votes_up").desc())
)

top_review_per_game_df = (
    reviews
    .filter(F.col("weighted_vote_score").isNotNull())
    .withColumn("influence_rank", F.rank().over(
   influence_window))
    .filter(F.col("influence_rank") == 1)
    .select("appid", "steamid", "voted_up", "
   weighted_vote_score",
            "votes_up", "playtime_forever", "influence_rank")
)
print(f"Games with top review: {top_review_per_game_df.count()
   :,}")
```

Spark SQL

```
SELECT appid, steamid, voted_up,
       weighted_vote_score, votes_up,
       playtime_forever, influence_rank
FROM (
    SELECT appid, steamid, voted_up,
           weighted_vote_score, votes_up, playtime_forever,
           RANK() OVER (
               PARTITION BY appid
               ORDER BY weighted_vote_score DESC,
                        votes_up DESC
           ) AS influence_rank
```

```
    FROM   reviews
    WHERE  weighted_vote_score IS NOT NULL
) ranked
WHERE influence_rank = 1;
```

RDD API

```
top_review_rdd = (
    reviews_rdd
    .filter(lambda r: r["weighted_vote_score"] is not None)
    .map(lambda r: (r["appid"], r))
    .groupByKey()
    .mapValues(lambda reviews: sorted(
        reviews,
        key=lambda r: (-float(r["weighted_vote_score"] or 0),
                       -int(r["votes_up"] or 0))
    )[0])
    .map(lambda kv: kv[1])
)
print(f"Games with top review: {top_review_rdd.count():,}")
```

Query 6 — Window Function: 7-Day Moving Average

Goal: Compute a 7-day rolling average of daily review volume per game to surface trend momentum.

Type: Window Function (Moving Average)

DataFrame API

```
daily_counts_df = (
    reviews
    .withColumn("review_date",
                F.to_date(F.to_timestamp("
   unix_timestamp_created")))
    .filter(F.col("review_date").isNotNull())
    .groupBy("appid", "review_date")
    .agg(F.count("*").alias("daily_review_count"))
)

hype_window = (
    Window
    .partitionBy("appid")
```

```
    .orderBy(F.col("review_date").cast("long"))
    .rangeBetween(-(6 * 86_400), 0)
)

hype_trend_df = (
    daily_counts_df
    .withColumn("avg_7d_reviews",
                F.round(F.avg("daily_review_count").over(
   hype_window), 2))
    .orderBy("appid", "review_date")
)
hype_trend_df.show(15, truncate=False)
```

Spark SQL

```
WITH daily AS (
    SELECT appid,
           TO_DATE(TO_TIMESTAMP(unix_timestamp_created)) AS
   review_date,
           COUNT(*) AS daily_review_count
    FROM   reviews
    WHERE  unix_timestamp_created IS NOT NULL
    GROUP  BY appid, TO_DATE(TO_TIMESTAMP(
   unix_timestamp_created))
)
SELECT appid, review_date, daily_review_count,
       ROUND(AVG(daily_review_count) OVER (
           PARTITION BY appid
           ORDER BY CAST(review_date AS LONG)
           RANGE BETWEEN 518400 PRECEDING AND CURRENT ROW
       ), 2) AS avg_7d_reviews
FROM   daily
ORDER  BY appid, review_date;
```

RDD API

```
from datetime import datetime, timedelta

daily_rdd = (
    reviews_rdd
    .filter(lambda r: r["unix_timestamp_created"] is not None)
```

```
    .map(lambda r: (
        (r["appid"],
         datetime.utcfromtimestamp(int(r["
   unix_timestamp_created"]))
                .strftime("%Y-%m-%d")), 1))
    .reduceByKey(lambda a, b: a + b)
    .map(lambda kv: (kv[0][0], (kv[0][1], kv[1])))
    .groupByKey()
    .flatMap(lambda kv: compute_7d_avg(kv[0], list(kv[1])))
)
```

Query 7 — Subquery / Nested Query

Goal: Return elite reviews that belong to games whose average helpfulness exceeds the global average — a two-phase filter requiring a nested subquery.

Type: Nested Subquery

Result: Global average = 0.1800; 536 elite reviews in above-average games (65.8% of all elite reviews).

DataFrame API

```
# Phase 1: compute global average as Python scalar
global_avg = (
    reviews
    .filter(F.col("weighted_vote_score").isNotNull())
    .agg(F.avg("weighted_vote_score"))
    .collect()[0][0]
)

# Phase 2: per-game averages above global
per_game_avg_df = (
    reviews
    .filter(F.col("weighted_vote_score").isNotNull())
    .groupBy("appid")
    .agg(
        F.round(F.avg("weighted_vote_score"), 4).alias("
   game_avg_score"),
        F.count("*").alias("game_review_count"),
    )
    .filter((F.col("game_review_count") >= 20)
            & (F.col("game_avg_score") > global_avg))
)
```

```python
# Phase 3: join with cached elite_df
above_avg_elite_df = (
    elite_df
    .join(per_game_avg_df, on="appid", how="inner")
    .select("steamid", "appid", "weighted_vote_score",
            "game_avg_score", "playtime_forever")
)
print(f"Matching reviews: {above_avg_elite_df.count():,}")
```

Spark SQL

```sql
SELECT e.steamid, e.appid, e.weighted_vote_score,
       g.game_avg_score, e.playtime_forever
FROM   elite_reviews e
INNER JOIN (
    SELECT appid,
           ROUND(AVG(weighted_vote_score), 4) AS game_avg_score
   ,
           COUNT(*) AS game_review_count
    FROM   reviews
    WHERE  weighted_vote_score IS NOT NULL
    GROUP  BY appid
    HAVING COUNT(*) >= 20
       AND ROUND(AVG(weighted_vote_score), 4) > (
           SELECT AVG(weighted_vote_score)
           FROM   reviews
           WHERE  weighted_vote_score IS NOT NULL
       )
) g ON e.appid = g.appid;
```

RDD API

```python
# Compute global average as Python scalar
wvs_rdd = (reviews_rdd
    .filter(lambda r: r["weighted_vote_score"] is not None)
    .map(lambda r: float(r["weighted_vote_score"])))
total    = wvs_rdd.sum()
count    = wvs_rdd.count()
global_avg = total / count
```

```
# Per-game averages
game_avg_rdd = (
    reviews_rdd
    .filter(lambda r: r["weighted_vote_score"] is not None)
    .map(lambda r: (r["appid"],
                    (float(r["weighted_vote_score"]), 1)))
    .reduceByKey(lambda a, b: (a[0]+b[0], a[1]+b[1]))
    .filter(lambda kv: kv[1][1] >= 20
                       and kv[1][0]/kv[1][1] > global_avg)
    .map(lambda kv: (kv[0], round(kv[1][0]/kv[1][1], 4)))
)

above_avg_set = set(game_avg_rdd.keys().collect())
result_rdd = (
    elite_rdd
    .filter(lambda r: r["appid"] in above_avg_set)
)
print(f"Matching reviews: {result_rdd.count():,}")
```

Query 8 — Broadcast Join

Goal: Enrich the 1.46M reviews with game names and genres from a small lookup table without triggering a shuffle on the large side.
Type: Join Optimization (Broadcast Join)
Confirmed: Physical plan shows `BroadcastHashJoin` with `BroadcastExchange` on the metadata side only.

DataFrame API

```
app_metadata_df = spark.createDataFrame([
    (10,      "Counter-Strike",    "Action"),
    (570,     "Dota 2",            "Strategy"),
    (730,     "CS:GO",             "FPS"),
    (578080,  "PUBG",              "Battle Royale"),
    (1172620, "Sea of Thieves",    "Adventure"),
    (1091500, "Cyberpunk 2077",    "RPG"),
    (945360,  "Among Us",          "Social Deduction"),
    (1422450, "Vampire Survivors", "Rogue-like"),
], schema=APP_META_SCHEMA)

broadcast_joined_df = (
    reviews
```

```
    .join(F.broadcast(app_metadata_df), on="appid", how="inner"
   )
    .select("appid", "app_name", "genre",
            "voted_up", "weighted_vote_score", "
   playtime_forever")
)
broadcast_joined_df.explain(True)
broadcast_joined_df.show(10, truncate=False)
```

Spark SQL

```
SELECT /*+ BROADCAST(a) */
       r.appid, a.app_name, a.genre,
       r.voted_up, r.weighted_vote_score, r.playtime_forever
FROM   reviews r
INNER JOIN app_metadata a ON r.appid = a.appid;
```

RDD API

```
# Broadcast the metadata dictionary to all executors
meta_dict = {10: ("Counter-Strike","Action"),
             570: ("Dota 2","Strategy"),
             730: ("CS:GO","FPS"), ...}
meta_bc = sc.broadcast(meta_dict)

broadcast_rdd = (
    reviews_rdd
    .filter(lambda r: r["appid"] in meta_bc.value)
    .map(lambda r: {
        "appid":     r["appid"],
        "app_name":  meta_bc.value[r["appid"]][0],
        "genre":     meta_bc.value[r["appid"]][1],
        "voted_up":  r["voted_up"],
        "wvs":       r["weighted_vote_score"],
        "playtime":  r["playtime_forever"],
    })
)
for row in broadcast_rdd.take(10):
    print(row)
```

Query 9 — Partition Pruning

Goal: Analyse monthly review volumes and average scores for 2019, demonstrating how a year filter reduces I/O by pushing the predicate into the Parquet scan.
Type: Optimization — Partition Pruning / Predicate Pushdown
Key observation: Physical plan shows the year computation pushed into `FileScan` `DataFilters`, and only 2 of 13 columns are read.

DataFrame API

```
pruned_df = (
    reviews
    .filter(F.col("review_year") == 2019)
    .groupBy("review_year", "review_month")
    .agg(
        F.count("*").alias("monthly_reviews"),
        F.round(F.avg("weighted_vote_score"), 4).alias("
   avg_score"),
    )
    .orderBy("review_year", "review_month")
)
pruned_df.explain(True)
pruned_df.show(12, truncate=False)
```

Spark SQL

```
SELECT review_year, review_month,
       COUNT(*)                          AS monthly_reviews,
       ROUND(AVG(weighted_vote_score),4) AS avg_score
FROM   reviews
WHERE  review_year = 2019
GROUP  BY review_year, review_month
ORDER  BY review_year, review_month;
```

RDD API

```
from datetime import datetime

pruned_rdd = (
    reviews_rdd
    .filter(lambda r: r["unix_timestamp_created"] is not None)
    .filter(lambda r:
```

```
        datetime.utcfromtimestamp(
            int(r["unix_timestamp_created"])).year == 2019)
    .map(lambda r: (
        (datetime.utcfromtimestamp(
             int(r["unix_timestamp_created"])).year,
         datetime.utcfromtimestamp(
             int(r["unix_timestamp_created"])).month),
        (float(r["weighted_vote_score"] or 0), 1)
    ))
    .reduceByKey(lambda a, b: (a[0]+b[0], a[1]+b[1]))
    .map(lambda kv: {
        "year":  kv[0][0], "month": kv[0][1],
        "count": kv[1][1],
        "avg":   round(kv[1][0]/kv[1][1], 4)
    })
    .sortBy(lambda r: (r["year"], r["month"]))
)
for row in pruned_rdd.collect():
    print(row)
```

Query 10 — Sort-Merge Join

Goal: Join reviews with user profile data on `steamid`, demonstrating a forced sort-merge join for large-vs-large join scenarios.

Type: Join Optimization (Sort-Merge Join)

Note: `autoBroadcastJoinThreshold` was set to `-1` to disable automatic broadcast upgrade and force the two-shuffle sort-merge pattern.

DataFrame API

```
# Force sort-merge join by disabling broadcast threshold
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "-1")

reviews_pre_partitioned = reviews.repartition(200, "steamid")

sort_merge_joined_df = (
    reviews_pre_partitioned
    .join(user_profiles_df, on="steamid", how="inner")
    .select("steamid", "username", "account_age_days",
            "total_reviews", "appid", "voted_up",
            "weighted_vote_score", "playtime_forever")
    .withColumn("is_veteran",
```

```
                F.col("account_age_days") >= 365)
)
sort_merge_joined_df.explain(True)
sort_merge_joined_df.show(15, truncate=False)

# Restore threshold
spark.conf.set("spark.sql.autoBroadcastJoinThreshold",
               str(50 * 1024 * 1024))
```

Spark SQL

```
-- Disable broadcast for this query
-- (autoBroadcastJoinThreshold=-1 set before executing)
SELECT r.steamid, u.username, u.account_age_days,
       u.total_reviews, r.appid, r.voted_up,
       r.weighted_vote_score, r.playtime_forever,
       (u.account_age_days >= 365) AS is_veteran
FROM   reviews r
INNER JOIN user_profiles u ON r.steamid = u.steamid;
```

RDD API

```
# RDD .join() always performs a shuffle-based join ---
# equivalent to sort-merge. No optimizer exists in RDD API.
reviews_keyed   = reviews_rdd.map(lambda r: (r["steamid"], r))
profiles_keyed  = profiles_rdd.map(lambda p: (p["steamid"], p))

joined_rdd = (
    reviews_keyed
    .join(profiles_keyed)
    .map(lambda kv: {
        "steamid":     kv[0],
        "username":    kv[1][1]["username"],
        "account_age": kv[1][1]["account_age_days"],
        "appid":       kv[1][0]["appid"],
        "voted_up":    kv[1][0]["voted_up"],
        "wvs":         kv[1][0]["weighted_vote_score"],
        "veteran":     int(kv[1][1]["account_age_days"]) >=
   365,
    })
)
```

```
for row in joined_rdd.take(15):
    print(row)
```

Execution Plan Comparison

How Spark Processes a Query: Three Plan Stages

When a DataFrame or SQL query is submitted to Spark, it passes through three sequential plan representations before any data is read. The **Parsed Logical Plan** captures the user's raw intent with unresolved column references. The **Optimized Logical Plan** reflects the transformations applied by Catalyst — filter fusion, predicate pushdown, constant folding, and column pruning. The **Physical Plan** is the final execution strategy specifying concrete operators, exchange (shuffle) nodes, and file scan strategies. The RDD API bypasses all three stages entirely, executing as written with no optimization layer.

Query 1 — Complex Filter

Parsed Logical Plan: The four `.filter()` calls appear as four stacked `Filter` nodes in sequence, each wrapping the next. Column references such as `playtime_forever` and `review` remain unresolved at this stage.

Optimized Logical Plan: Catalyst performs two critical transformations. First, all four filter predicates are *fused* into a single compound `Filter` node — `(playtime_forever >= 3000) AND (weighted_vote_score > 0.7) AND voted_up AND (length(review) >= 100)`. Second, Catalyst automatically injects `isnotnull` checks for each referenced column, eliminating the need for explicit null guards in application code.

Physical Plan: The `FileScan parquet` node reveals two Parquet-level optimizations in its attributes. The `PushedFilters` field confirms that `IsNotNull(playtime_forever)`, `IsNotNull(weighted_vote_score)`, `IsNotNull(voted_up)`, and `IsNotNull(review)` are pushed into the Parquet reader — row groups that cannot satisfy these predicates are skipped before data enters JVM memory. The `ReadSchema` field shows only 8 of 13 columns are read from disk, confirming column pruning.

Key Insight: Catalyst not only fuses four sequential filter calls into one, but also delegates numeric predicates to the Parquet row-group level, meaning large portions of the file are never read. This is the central advantage of structured APIs over the RDD API, which must deserialize every row regardless of whether it survives the filter.

Query 2 — Aggregation

Parsed Logical Plan: Shows an `UnresolvedHaving` node wrapping an `Aggregate` on `appid` with four aggregate functions. The `HAVING COUNT(*) >= 50` clause appears as an unresolved predicate because the alias `total_reviews` does not yet exist at parse time —

it is referenced by name before the Analyzer has resolved it.

Optimized Logical Plan: Catalyst performs two key transformations. First, the `HAVING` filter becomes a post-aggregate `Filter (total_reviews >= 50)` node. Second, column pruning reduces the `FileScan` to only 4 of 13 columns: `appid`, `votes_up`, `weighted_vote_score`, and `playtime_forever`. All other columns are eliminated because the aggregation does not reference them.

Physical Plan: Shows the classic two-phase partial aggregation pattern. A `HashAggregate` (partial) runs locally on each executor computing partial sums and counts per `appid` partition, then an `Exchange hashpartitioning(appid, 200)` shuffles partial results so all rows for the same game land on the same executor, and a final `HashAggregate` merges them. The `FileScan` confirms only 4 columns are read with no `PushedFilters` — there is no filter predicate to push down since all rows contribute to the aggregation.

Key Insight: Two-phase partial aggregation is the most important optimization for groupBy queries. Pre-aggregating on each executor before the shuffle means only one partial sum per `appid` per executor crosses the network, not one row per review. For a dataset with 976,212 CS:GO reviews, this reduces shuffle volume by orders of magnitude.

Query 3 — Multi-Attribute Grouping

Parsed Logical Plan: Shows two stacked `Filter` nodes — `isnotnull(voted_up)` and `isnotnull(appid)` — followed by an `Aggregate` on the compound key `[appid, voted_up]`, then a `Sort` on `review_count DESC`.

Optimized Logical Plan: Catalyst fuses the two null checks into a single compound filter `(isnotnull(voted_up) AND isnotnull(appid))` and pushes it directly onto the `Relation`. Column pruning reduces the scan to 4 columns: `appid`, `voted_up`, `weighted_vote_score`, and `playtime_forever`. The `PushedFilters` field in the Physical Plan confirms both null checks are pushed to the Parquet reader level.

Physical Plan: Shows two `Exchange` nodes — the first is `hashpartitioning(appid, voted_up, 200)` for the groupBy shuffle, and the second is `rangepartitioning(review_count DESC, 200)` for the global sort. The sort requires a second shuffle because the partition key for aggregation differs from the sort key. The two-phase `HashAggregate` pattern is again visible, with `partial_stddev` computed locally before the shuffle.

Key Insight: A multi-key groupBy produces a wider shuffle key than a single-key groupBy, but Catalyst still applies the same partial pre-aggregation optimization. The additional `rangepartitioning` shuffle for the global `ORDER BY` is unavoidable — global ordering requires a single global sort pass across all executors.

Query 4 — Sorting and Ranking

Parsed Logical Plan: Shows `GlobalLimit 10` wrapping `LocalLimit 10` wrapping a `Sort` — this is how Spark represents `ORDER BY ... LIMIT` before optimization. The engagement score formula `total_reviews * avg_helpfulness * LOG1P(avg_playtime_mins)` appears as an expression referencing the `per_game_stats` intermediate result.

Optimized Logical Plan: Catalyst folds constant arithmetic — the integer literal `1` in `LN(avg_playtime_mins + 1)` is promoted to `1.0` at plan time. Column pruning reduces the underlying scan to 4 columns. The `HAVING` filter `total_reviews >= 50` is retained as a post-aggregate filter node.

Physical Plan: The `GlobalLimit + LocalLimit + Sort` combination is rewritten as a single `TakeOrderedAndProject` operator — the canonical top-K optimization. Instead of sorting all rows globally and then taking 10, `TakeOrderedAndProject` maintains a local heap of 10 rows per executor, discarding all others. Only 10 rows per executor travel to the driver for the final merge, regardless of how many distinct games exist in the dataset.

Key Insight: A naive implementation would sort all games globally (requiring a full shuffle), then limit to 10. The `TakeOrderedAndProject` heap-based approach eliminates the global sort shuffle entirely — executor-local heaps are merged at the driver with no network data movement beyond the top-10 candidates per executor.

Query 6 — Moving Average

Parsed Logical Plan: The DataFrame version shows a `Window` node with `rangeBetween(-6, 0)` over `unix_date(review_date)`, pre-aggregated by `[appid, review_date]`. The SQL version shows a `CTE [daily_counts]` definition followed by a `Window` referencing it — the CTE appears as a `CTERelationDef` and `CTERelationRef` pair.

Optimized Logical Plan: Catalyst inlines the CTE, replacing the `CTERelationRef` with the CTE body directly and eliminating the indirection. The `unix_date()` computation is pushed into the aggregation key as `_groupingexpression`, computing it once during the daily count aggregation rather than separately. The `FileScan` is reduced to just 2 of 13 columns: `appid` and `unix_timestamp_created` — an 85% reduction in I/O.

Physical Plan: Reveals a three-shuffle pipeline: `hashpartitioning(appid, review_date)` for the daily count aggregation, `hashpartitioning(appid)` for the window function partition, and `rangepartitioning(appid, review_date)` for the final sort. The window function executes as a `RangeFrame` with bounds `-6 to currentrow()` operating on `unix_date` integer values — a 7-day window using day-number arithmetic.

Key Insight: Pre-aggregating to daily counts before applying the window is critical. Running a range window directly on 1.46M raw rows would require holding the entire per-game review history in executor memory during the window pass. By reducing to one

row per (game, day) first, the window input is orders of magnitude smaller. CTE inlining by Catalyst also ensures the daily aggregation is computed only once even though it is logically referenced twice in the query.

Query 5 — Window Function Ranking

Parsed Logical Plan: Shows a `Window` node computing `rank()` over `(appid, weighted_vote_score DESC, votes_up DESC)`, followed by a separate `Filter` node keeping only `influence_rank = 1`.

Optimized Logical Plan: Catalyst inserts a `WindowGroupLimit` node between the data scan and the `Window` computation. This is a significant optimization: rather than computing the rank for every row across all 1.46M reviews and then discarding all but rank-1 rows, Catalyst injects a pre-filter that keeps only the top-1 candidate per `appid` partition *before* the shuffle. This dramatically reduces the volume of data that crosses executor boundaries.

Physical Plan: Reveals a two-phase execution of `WindowGroupLimit`: a `Partial` phase runs locally on each executor before the `Exchange hashpartitioning(appid, 200)` shuffle, and a `Final` phase runs after. This mirrors the two-phase partial aggregation pattern used for `groupBy`.

Key Insight: Without `WindowGroupLimit`, Spark would shuffle all 1.46M rows by `appid`, compute ranks globally, then filter. With it, most rows are eliminated locally, and only the top candidates per partition are shuffled — a substantial reduction in network I/O.

Query 7 — Subquery

Parsed Logical Plan: Shows a `Join` between the `elite_reviews` view and an `UnresolvedHaving` subquery that itself contains a `scalar-subquery` computing the global average.

Optimized Logical Plan: Two critical optimizations are visible. First, the `elite_reviews` side of the join resolves to an `InMemoryRelation` — confirming that the `cache()` call from Q1 was honored and the full Parquet scan is avoided. Second, the global average subquery reads only the `weighted_vote_score` column (`ReadSchema: struct<weighted_vote_score:float>`), demonstrating aggressive column pruning on the subquery branch.

Physical Plan: The join between `elite_df` (814 cached rows) and the per-game aggregation results is executed as a `BroadcastHashJoin` with `BuildLeft` — Spark correctly identifies the cached elite table as the smaller side and broadcasts it. The scalar subquery appears as a separate `Subquery` branch with its own `HashAggregate` pipeline.

Key Insight: Caching `elite_df` after Q1 converts what would have been a second full Parquet scan into an `InMemoryTableScan`. In iterative analytical workflows where intermediate results are reused, caching is essential and Catalyst explicitly acknowledges

it in the plan.

Query 8 — Broadcast Join

Parsed Logical Plan: The DataFrame version shows an `UnresolvedHint BROADCAST` on the `app_metadata` relation. The SQL version shows `ResolvedHint (strategy=broadcast)` after the analyzer resolves the `/*+ BROADCAST(a) */` comment.

Optimized Logical Plan: The hint is preserved as `rightHint=(strategy=broadcast)` on the join node. Catalyst does not discard user hints — it incorporates them as constraints on the physical planning phase.

Physical Plan: Confirms `BroadcastHashJoin [appid], [appid], Inner, BuildRight` — the metadata table is the build side. A `BroadcastExchange HashedRelationBroadcastMode` node appears only on the metadata side; the reviews side (`FileScan parquet`) has no corresponding `Exchange` node, confirming that the 1.46M-row table does not shuffle. The 8-row metadata table is serialized into a hash map and sent to every executor.

Key Insight: In a sort-merge join both sides would require an `Exchange hashpartitioning` node — the 1.46M reviews would be shuffled across the network by `appid`. The broadcast eliminates this entirely: only 8 rows cross executor boundaries.

Query 9 — Partition Pruning

Parsed Logical Plan: Shows a `Filter` on `review_year IN (2019)` applied over a view that derives `review_year` from `unix_timestamp_created` using `year(to_timestamp(...))`. At parse time this is just a filter on a derived column.

Optimized Logical Plan: Catalyst pushes the derived year filter back through the projection layers and onto the raw `unix_timestamp_created` column: `year(cast(cast(unix_timestamp_crea` `as timestamp) as date)) = 2019`. The `ReadSchema` in this plan shows only `weighted_vote_score` and `unix_timestamp_created` — 2 of 13 columns — are needed.

Physical Plan: The `FileScan parquet` node shows the year predicate in `DataFilters`, confirming it is evaluated during the scan. `PartitionFilters` is empty because the data was not physically written with `partitionBy("review_year")` — this is filter pushdown to row-group level, not directory-level partition skipping. Had the data been partitioned on disk, the `PartitionFilters` field would carry the predicate and entire directories would be skipped.

Key Insight: Even when a filter references a *derived* column (year computed from a Unix timestamp), Catalyst traces the computation back to its source column and pushes the predicate into the Parquet reader. This is possible because Catalyst's optimizer understands the full lineage of every column expression.

Query 10 — Sort-Merge Join

Parsed Logical Plan: Shows a `RepartitionByExpression [steamid], 200` node wrapping the reviews relation, followed by a `Join Inner` on `steamid`.

Optimized Logical Plan: Catalyst preserves the explicit repartition (it cannot eliminate a user-specified repartition) and pushes an `isnotnull(steamid)` filter down to the scan. Column pruning reduces the read to 5 of 13 columns.

Physical Plan: Confirms `SortMergeJoin [steamid#0L], [steamid#19L], Inner` — the two-shuffle sort-merge pattern. Two `Exchange hashpartitioning(steamid, 200)` nodes are visible, one per join side. The reviews side is labeled `REPARTITION_BY_NUM` (from the explicit `.repartition(200, "steamid")` call), while the user_profiles side is labeled `ENSURE_REQUIREMENTS`. Setting `autoBroadcastJoinThreshold=-1` was necessary because AQE would otherwise have detected the small user_profiles table at runtime and silently upgraded the join to a BroadcastHashJoin.

Key Insight: AQE's runtime re-optimization is powerful but can obscure educational demonstrations. The explicit threshold override is the correct mechanism to force a specific join strategy for analysis or testing purposes.

RDD API: No Execution Plans

The RDD API completely bypasses the Catalyst optimizer. When an RDD transformation chain is executed, Spark creates a *lineage graph* (DAG) of RDD dependencies, but there is no logical plan, no optimization pass, and no physical plan in the Catalyst sense. Consequences include:

- No filter fusion — four `.filter()` calls become four separate passes over the data
- No predicate pushdown — the Parquet reader deserializes all rows and all columns regardless of what the query needs
- No column pruning — every field in every row is deserialized into a Java object even if only one field is used
- No join optimization — `rdd.join()` always performs a full shuffle sort-merge regardless of table sizes
- No caching awareness in the plan — caching exists but is not reflected in any optimization pass

This is the fundamental reason why Q10 in RDD took 59.1 seconds versus 3.5 seconds in DataFrame — a 17x performance gap attributable entirely to the absence of optimization.

SQL vs DataFrame: Identical Optimized Plans

Spark SQL and the DataFrame API produce *identical* Optimized Logical Plans and Physical Plans for equivalent queries. Both pass through the same Catalyst pipeline. The only structural difference is in the Parsed Logical Plan: SQL queries show `UnresolvedRelation` nodes (table names as strings awaiting the Analyzer), `UnresolvedAttribute` column references, and `UnresolvedHint` nodes. DataFrame queries show the programmatic construction directly — `Project`, `Filter`, and `Aggregate` nodes are already partially resolved at parse time because the API interacts with the schema at code-writing time rather than at query-string parse time.

Performance Comparison

Main Query Execution Times

All three API implementations were executed sequentially on the same GitHub Codespaces VM (2 vCPU, 8 GB RAM, Ubuntu 24) using the same Parquet dataset. Timings were measured with `time.perf_counter()` wall-clock precision.

Table 2: Query Execution Time by API (seconds)

Query	RDD (s)	DataFrame (s)	SQL (s)	Fastest
Q1 — Complex Filter	25.457	5.295	4.612	SQL
Q2 — Aggregation	21.720	1.540	1.409	SQL
Q3 — Multi-Group	0.569	1.518	1.655	RDD
Q4 — Sort & Rank	0.497	1.308	0.769	RDD
Q5 — Window Rank	24.916	2.262	2.072	SQL
Q6 — Moving Average	1.015	2.361	2.863	RDD
Q7 — Subquery	24.253	2.122	1.331	SQL
Q8 — Broadcast Join	4.082	3.423	2.019	SQL
Q9 — Partition Pruning	0.545	0.777	0.887	RDD
Q10 — Sort-Merge Join	59.099	3.537	6.974	DataFrame

Analysis

SQL and DataFrame APIs consistently outperform RDD on computationally intensive queries. For Q1 (complex filter), SQL was 5.5x faster than RDD; for Q2 (aggregation), SQL was 15.4x faster; for Q5 (window rank), SQL was 12x faster; and for Q7 (subquery), SQL was 18.2x faster. These speedups originate directly from Catalyst's filter fusion and predicate pushdown (Q1), partial pre-aggregation before shuffle (Q2), `WindowGroupLimit` pre-filtering (Q5), and cache-aware planning (Q7). The Tungsten execution engine further contributes through off-heap memory management and whole-stage code generation, which converts the physical plan into a single tight JVM bytecode loop per stage.

The RDD API wins on Q3, Q4, Q6, and Q9 — all queries where the primary action involves either a cached result (Q3, Q4 build on per_game_stats which was already computed) or a very small output (Q9 processes only 12 monthly aggregations). In these cases, the overhead of Catalyst plan compilation and JIT warm-up in the structured APIs slightly exceeds the trivial computation cost, giving the low-overhead RDD a narrow edge.

Q10 is the starkest illustration of the RDD API's limitations: 59.1 seconds versus 3.5 seconds in DataFrame (17x slower). The RDD join shuffles the full 1.46M-row dataset with no pushdown, no column pruning, and full Java object deserialization on both sides. DataFrame's sort-merge join, while still performing two shuffles, benefits from Tungsten's binary format and pushed-down null filters that eliminate non-matching rows before the shuffle stage.

DataFrame and SQL timings are very close across all queries because they share the same Catalyst optimization pipeline and the same Tungsten runtime. Differences on the order of milliseconds to low single seconds reflect query compilation overhead and minor JVM non-determinism rather than architectural differences.

Scalability Test — Partition Count

To isolate the effect of shuffle partition count on query performance, AQE was disabled and three representative queries were executed across four partition settings on the same dataset.

Table 3: Execution Time vs Shuffle Partition Count (seconds, AQE disabled)

Partitions	Q2 Aggregation	Q3 Multi-Group	Q5 Window Rank
50	10.780	1.844	4.469
100	2.047	1.770	4.364
200	3.587	2.553	4.585
400	4.269	3.549	6.139
Best	**100**	**100**	**100**

100 partitions was optimal across all three queries for this 1.46M-row dataset on a 2-core VM. The pattern is instructive: at 50 partitions, Q2 suffered most (10.78 seconds) because the aggregation shuffle produced 50 tasks, leaving one of the two cores idle for much of the computation. At 100 partitions the workload was evenly distributed across both cores. Beyond 100, task scheduling overhead — the cost of creating, tracking, and completing each task — dominates the parallelism benefit. At 400 partitions, Q5's window rank took 6.1 seconds compared to 4.4 seconds at 100. Spark's default of 200 shuffle partitions is calibrated for larger clusters; for a 2-core VM on this dataset, 100 is the empirically optimal setting.

File Format Benchmark — CSV vs Parquet

To quantify the impact of file format on query performance, the same four benchmarks were run against both `reviews-1-115.csv` and `steam_reviews.parquet` using a single session with AQE disabled for fair comparison.

Table 4: CSV vs Parquet Performance Benchmark

Benchmark	CSV (s)	Parquet (s)	Speedup
Count	30.312	0.471	64.36×
Aggregation	11.694	3.218	3.63×
Complex Filter	5.704	1.857	3.07×
Column Scan	2.234	0.300	7.45×
Average			**19.6×**

Parquet was 19.6× faster on average. The Count benchmark produced the most dramatic speedup (64.36×) because Parquet stores row counts in the file footer metadata — Spark can answer `count()` by reading a few bytes of metadata rather than scanning any data rows. CSV has no such metadata and must parse every row. The Column Scan speedup (7.45×) demonstrates columnar storage: Parquet physically organizes data by column on disk, so a query requesting only `appid`, `weighted_vote_score`, and `playtime_forever` reads approximately 23% of the file; CSV must parse all 13 fields on every row even if only 3 are needed. The Complex Filter speedup (3.07×) reflects predicate pushdown into Parquet row groups: row groups whose statistics (min/max per column) cannot satisfy the filter are skipped entirely before entering JVM memory.

Cluster Configuration Profiles

Three SparkSession profiles were benchmarked to characterize the impact of memory and parallelism settings.

Table 5: Cluster Configuration Profiles and Cold-Start Count Timing

Profile	Master	Driver Mem	Shuffle Parts	Count Time
Default	local[2]	4g	200	7.474s
Low	local[1]	2g	50	0.652s
High	local[*]	6g	400	0.534s

The apparent superiority of Low and High profiles on the count benchmark is an artifact of JVM warm-up: the Default profile ran first on a cold JVM, incurring class loading and JIT compilation costs. In a true cold-start comparison, the High profile (`local[*]` uses all available cores, 6g driver memory) would lead. The Default profile represents a balanced configuration for a 2-core 8GB VM. The Low profile demonstrates that single-core

execution with 50 partitions is viable for lightweight operations but would degrade severely on shuffle-heavy queries.

Final Insights

Business and Analytical Conclusions

1. CS:GO Dominates Review Activity by an Extreme Margin

CS:GO (appid=730) produced 976,212 reviews with an engagement score of 2,062,600.35 — more than 15× the second-ranked game, Left 4 Dead 2 (appid=550) at 133,505. This reflects CS:GO's status as the most-reviewed game in Steam history and its free-to-play transition in 2018, which dramatically expanded its player base. For platform operators, this level of concentration suggests that recommendation algorithms must account for review volume bias when surfacing quality signals.

2. Negative Reviews Are More Peer-Validated Than Positive Ones

Q3 reveals that for CS:GO (appid=730), negative reviews (voted_up=False) carry an average helpfulness score of 0.2283, compared to 0.1854 for positive reviews — a 23% premium. This pattern is consistent across multiple games in the dataset. Negative reviews tend to be more specific, more detailed, and more frequently address concrete problems (bugs, pricing, anti-cheat systems) that the community validates as useful. For developers, this implies that negative reviews contain higher-signal actionable feedback than positive ones.

3. Elite Reviewers Are Rare and Concentrated in Quality Games

Only 814 of 1,460,621 reviews (0.056%) satisfy all four elite criteria simultaneously: 50+ hours played, helpfulness score ≥ 0.7, positive recommendation, and review text of 100+ characters. Of these 814 elite reviews, 536 (65.8%) belong to games whose average helpfulness score exceeds the global mean of 0.18. This confirms that deeply engaged, peer-validated reviewers disproportionately gravitate toward games that the broader community also rates as having high-quality review discourse — a quality signal that could be used to bootstrap trust scores for newer games with fewer reviews.

4. Review Activity Exhibits Strong Seasonal Spikes

Q9 shows that November 2019 generated 46,972 reviews and December 2019 generated 34,620 — approximately 4×–6× the volume of quieter months such as September (5,358) and August (6,140). This pattern aligns precisely with Steam's annual winter sale period, which drives massive new player acquisition and re-engagement of existing players. Game

publishers and marketing teams can use this temporal insight to time review-incentivizing events and developer responses around peak activity windows.

5. Dota 2 Represents Extreme Time Investment

Q3 shows Dota 2 (appid=570) positive reviewers averaging 139,852 minutes of playtime — equivalent to approximately 2,331 hours. This is 2× higher than CS:GO positive reviewers (66,580 minutes) and 13× higher than the global average playtime in the dataset. The MOBA genre's infinite skill ceiling and competitive ranking system create a player base that commits at an extraordinary level before writing reviews. This suggests that playtime-weighted review scoring models need genre-specific calibration.

6. Moving Averages Reveal Launch and Update Events

The 7-day moving average in Q6 shows Counter-Strike original (appid=10) experiencing its first cluster of reviews in November–December 2010, with daily counts averaging 2–3 before spiking to 4 on November 20. This pattern, replicated across multiple games in the full dataset, indicates that the moving average is an effective signal for detecting game launches, major updates, and promotional events without requiring explicit event metadata. A streaming extension of this pipeline could power real-time anomaly detection for sudden review volume surges.

Lessons Learned About Spark Optimization

1. Catalyst Filter Fusion Is Automatic and Consequential

Writing four separate `.filter()` calls is functionally identical to writing one compound predicate — Catalyst merges them into a single `Filter` node in the Optimized Logical Plan and automatically injects `isnotnull` checks. This means developers should not fear writing readable, chained filter logic for the sake of perceived performance. The optimizer handles consolidation. The resulting single fused filter is then pushed to the Parquet reader level, reducing I/O before any data enters JVM memory.

2. Parquet Is Not Optional for Production Spark Workloads

The 19.6× average speedup over CSV — and the extreme 64× speedup on count operations — makes CSV effectively unusable for iterative analytical workloads. The one-time cost of converting CSV to Parquet (`scripts/convert_to_parquet.py`) is amortized across every subsequent query execution. In a production pipeline with dozens of daily query runs, this conversion pays for itself within hours of deployment.

3. The RDD API Should Be Reserved for Non-Standard Operations

Q10 took 59.1 seconds in RDD versus 3.5 seconds in DataFrame — a 17× performance gap on identical hardware and data. The RDD API is appropriate for operations that cannot be expressed in the structured API (custom serialization formats, graph algorithms, highly stateful streaming) but should not be used for analytical queries where Catalyst optimization applies. The absence of predicate pushdown, column pruning, and join strategy optimization in the RDD layer makes it categorically inferior for this workload class.

4. AQE's Runtime Re-Optimization Must Be Understood, Not Just Trusted

Without explicitly setting `autoBroadcastJoinThreshold=-1`, Spark's Adaptive Query Execution silently upgraded the intended SortMergeJoin in Q10 to a BroadcastHashJoin at runtime, after detecting that the user_profiles table was small. This is generally beneficial in production, but it means that the physical plan observed during development may differ from the plan executed on a differently-sized dataset in a later run. Engineers must understand AQE's runtime decision boundaries to reason correctly about query behavior under changing data volumes.

5. Shuffle Partition Count Must Be Tuned Per Dataset and Cluster

Spark's default of 200 shuffle partitions is calibrated for clusters with many executors and large datasets. On a 2-core VM with 1.46M rows, 100 partitions was optimal — cutting Q2's aggregation time from 10.78 seconds (at 50 partitions) to 2.05 seconds. The rule of thumb is: target 2–4 partitions per available core, scaled by the data volume per partition (aim for 128–256 MB per partition after shuffle). Blindly accepting the default on a small single-machine deployment leaves significant performance on the table.

6. Caching Intermediate Results Has Compounding Benefits

Caching `elite_df` after Q1 converted what would have been a second full Parquet scan in Q7 into an `InMemoryTableScan`. The Optimized Logical Plan for Q7 explicitly shows `InMemoryRelation` as the source for the elite reviewer side of the join — confirming that Catalyst is aware of the cache and plans accordingly. In multi-query workflows where a filtered or aggregated intermediate result is reused across multiple downstream queries, caching that intermediate is one of the highest-leverage optimizations available with essentially zero implementation cost.